

Deep learning avec Pytorch

Partie 5 : Votre premier réseau

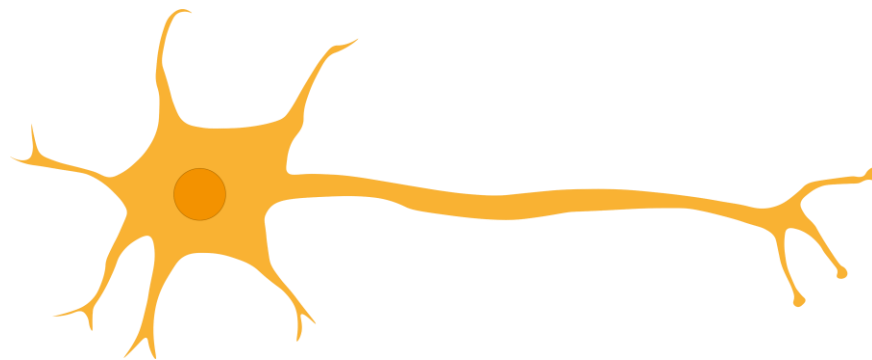


Présenté par **Morgan Gautherot**

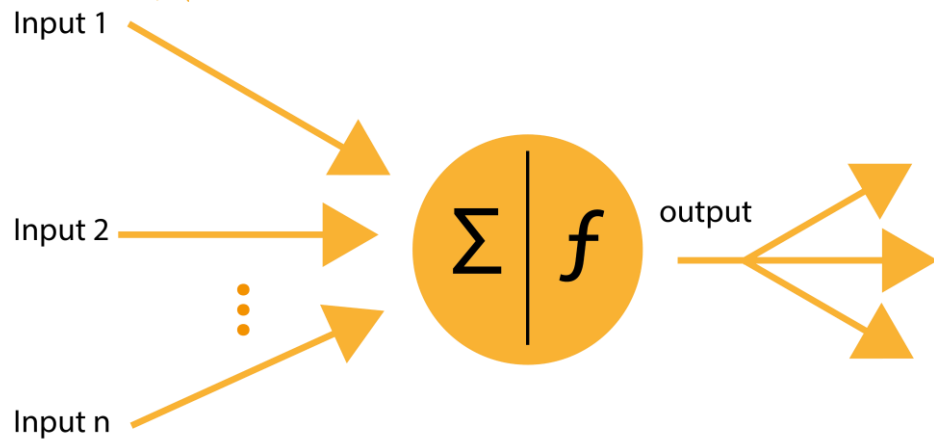


Le neurone artificiel

Neurone biologique

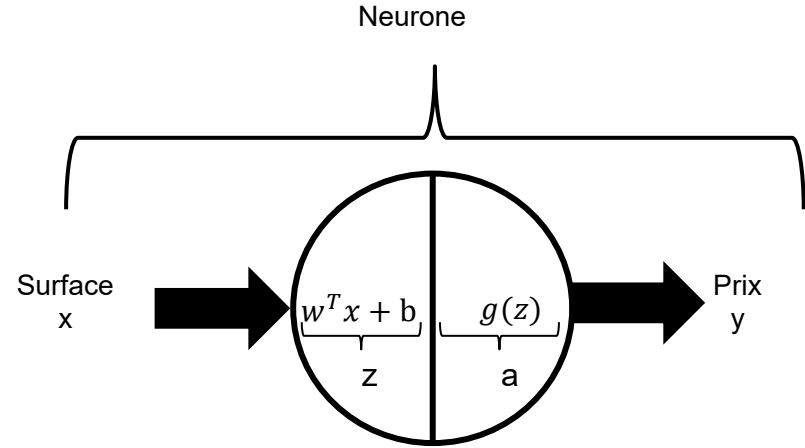
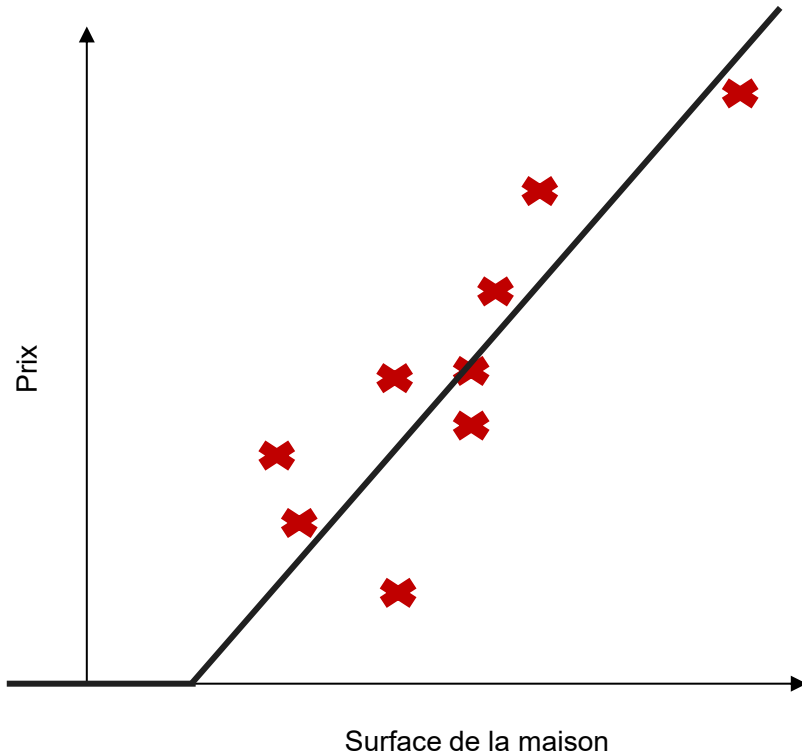


Neurone artificiel





Qu'est-ce qu'un réseau neuronal ?

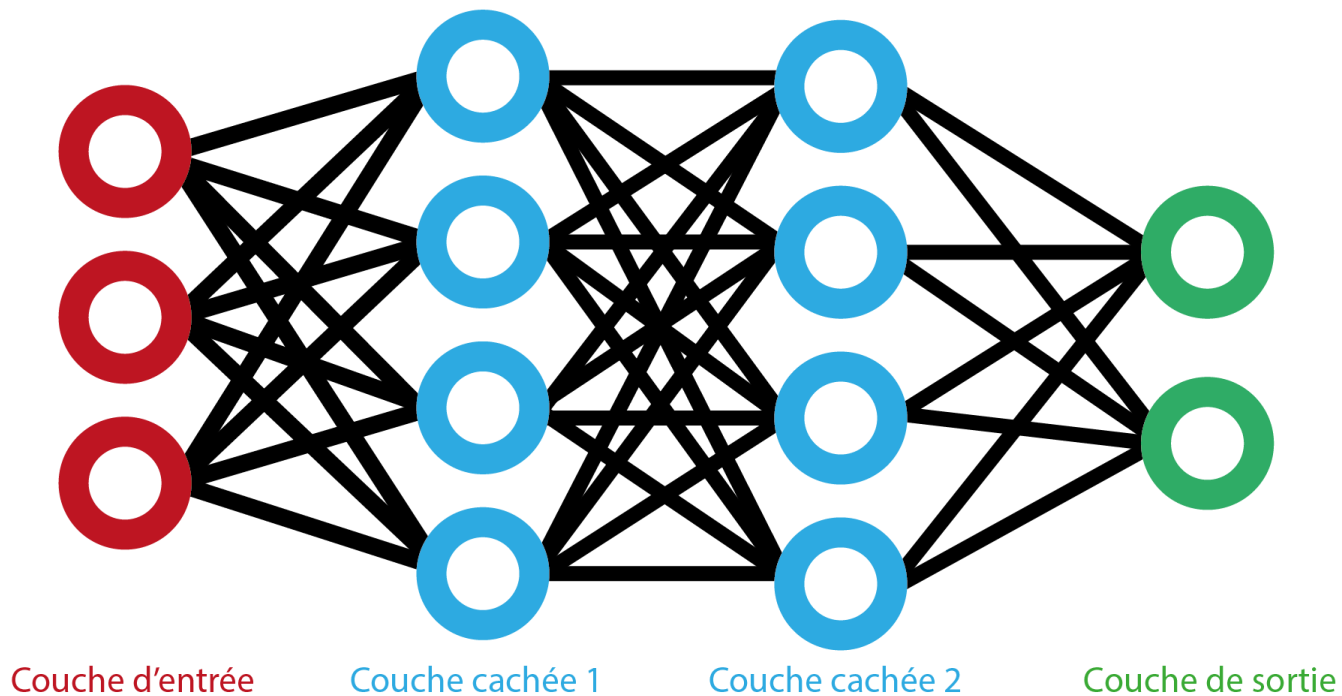


Un réseau neuronal est construit
en connectant de nombreux neurones

Fonction d'activation : $g(z) = \max(0, z)$

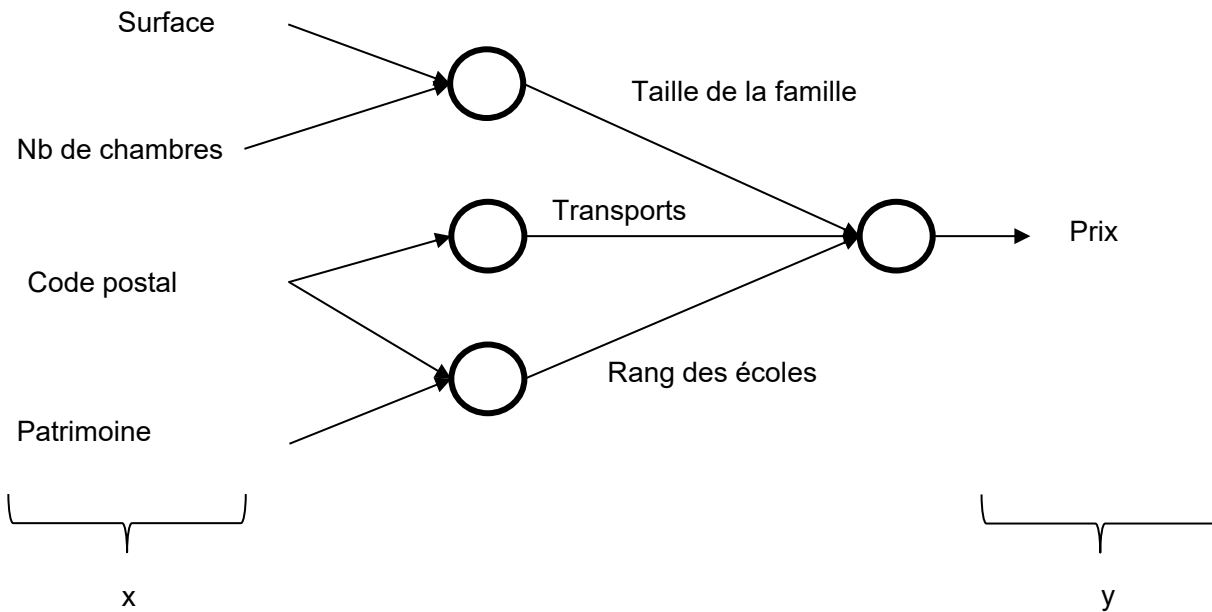


Réseau neuronal



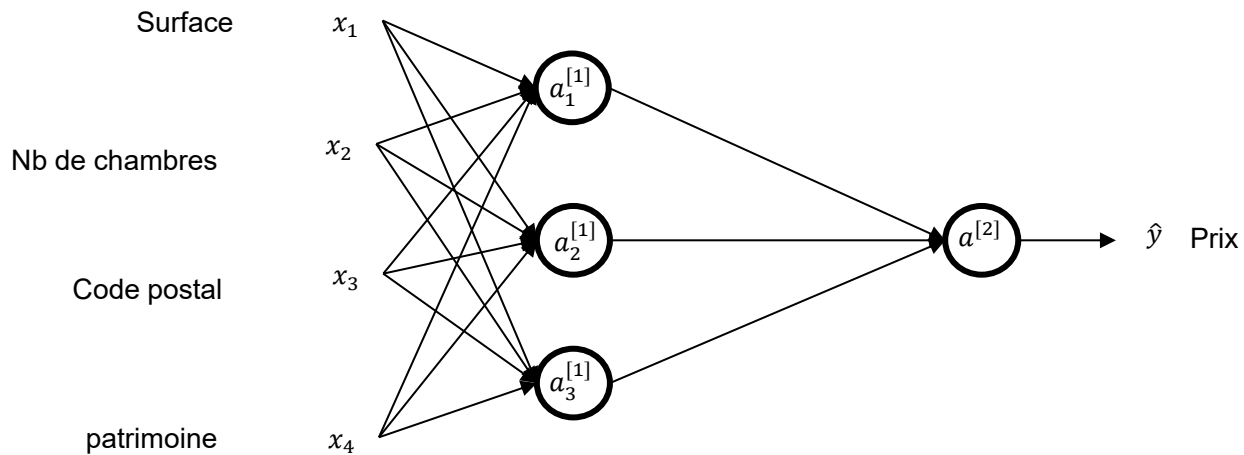


Le feature engineering





Feature engineering automatique



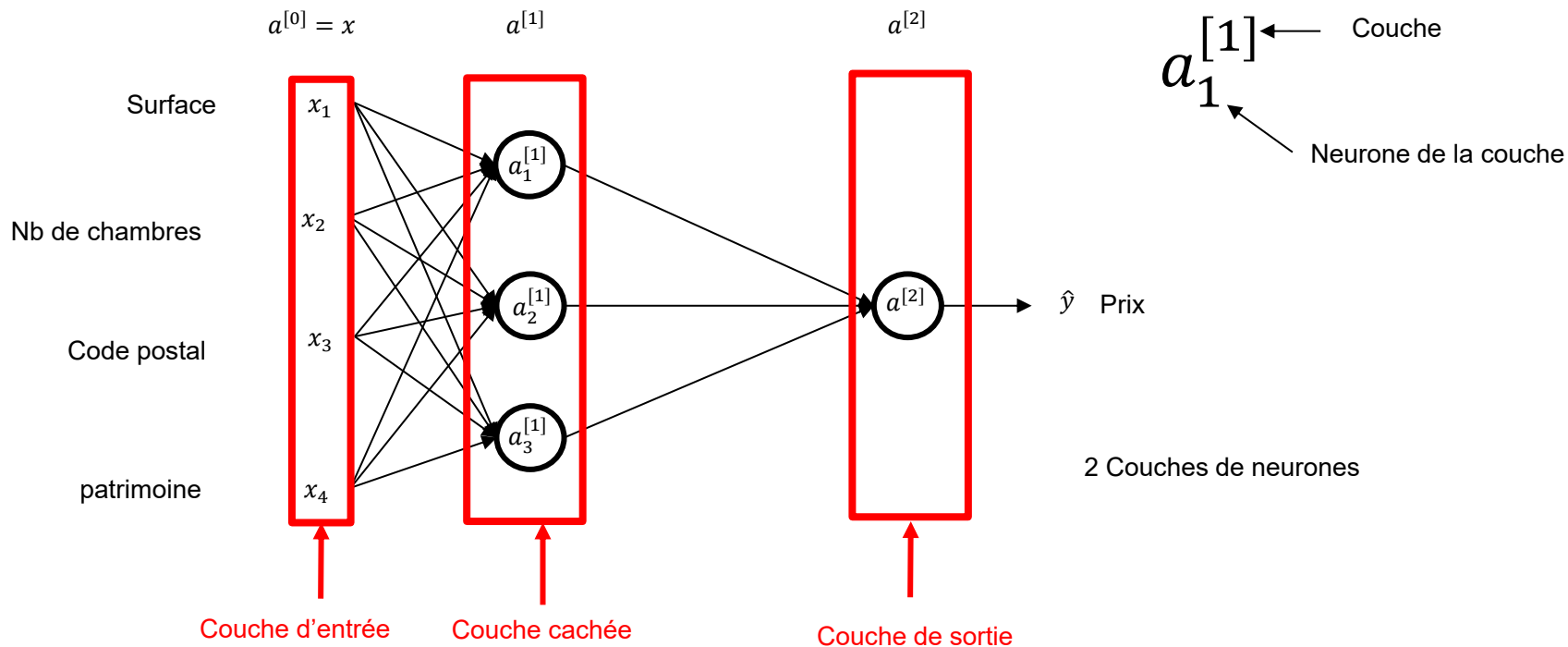
Représentation mathématique



Partie 5 : Votre premier réseau

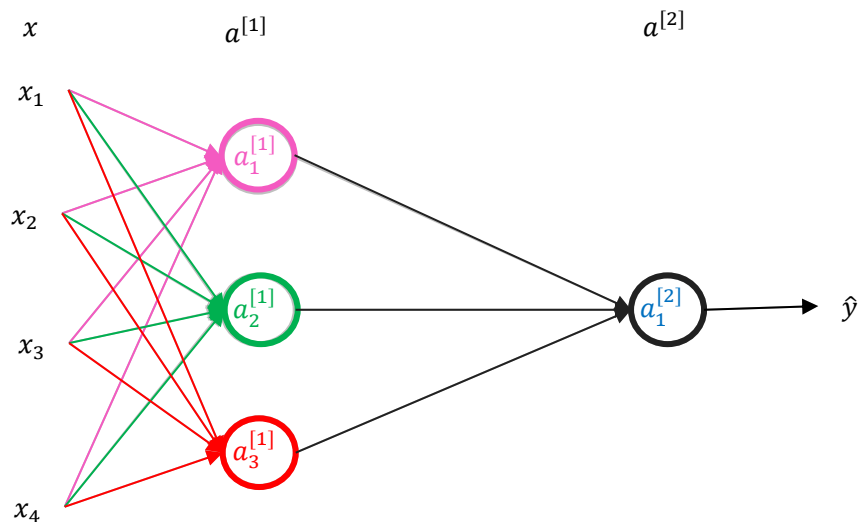


Un peu de notations





Et les mathématiques dans tout ça ?



$$z_1^{[1]} = w_1^{[1]T} x + b_1^{[1]} \quad ; \quad a_1^{[1]} = g(z_1^{[1]})$$

$$z_2^{[1]} = w_2^{[1]T} x + b_2^{[1]} \quad ; \quad a_2^{[1]} = g(z_2^{[1]})$$

$$z_3^{[1]} = w_3^{[1]T} x + b_3^{[1]} \quad ; \quad a_3^{[1]} = g(z_3^{[1]})$$

$$z_1^{[2]} = w_1^{[2]T} a^{[1]} + b_1^{[2]} \quad ; \quad a_1^{[2]} = g(z_1^{[2]})$$



Vectorisation sur un exemple

$$\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \end{bmatrix} = \begin{bmatrix} w_1^{[1]T} x + b_1^{[1]} \\ w_2^{[1]T} x + b_2^{[1]} \\ w_3^{[1]T} x + b_3^{[1]} \end{bmatrix} \quad ; \quad \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \end{bmatrix} = \begin{bmatrix} g(z_1^{[1]}) \\ g(z_2^{[1]}) \\ g(z_3^{[1]}) \end{bmatrix}$$

$$\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \end{bmatrix} = \begin{bmatrix} w_{1,1}^{[1]} & w_{1,2}^{[1]} & w_{1,3}^{[1]} & w_{1,4}^{[1]} \\ w_{2,1}^{[1]} & w_{2,2}^{[1]} & w_{2,3}^{[1]} & w_{2,4}^{[1]} \\ w_{3,1}^{[1]} & w_{3,2}^{[1]} & w_{3,3}^{[1]} & w_{3,4}^{[1]} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \end{bmatrix} \quad ; \quad \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \end{bmatrix} = g\left(\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \end{bmatrix}\right)$$

$$\begin{matrix} (3,1) & (3,4) & (4,1) & (3,1) \\ z^{[1]} & = & W^{[1]T} x & + & b^{[1]} \end{matrix} \quad ; \quad a^{[1]} = g(z^{[1]})$$

$$\begin{matrix} (1,1) & (1,3) & (3,1) & (1,1) \\ z^{[2]} & = & w^{[2]T} a^{[1]} & + & b^{[2]} \end{matrix} \quad ; \quad a^{[2]} = g(z^{[2]})$$



Vectorisation sur plusieurs exemples

for $i = 0$ to m :

$$a^{[1](i)} = g(w^{[1]T} x^{(i)} + b)$$

Diagram showing inputs $(n_1, 1)$, (n_1, n_x) , $(n_x, 1)$, and $(n_1, 1)$ feeding into the equation above.

$$\hat{y}^{(i)} = a^{[2](i)} = g(w^{[2]T} a^{[1](i)} + b)$$

Diagram showing inputs $(1, 1)$, $(1, n_1)$, $(n_1, 1)$, and $(1, 1)$ feeding into the equation above.

$$x \in \mathbb{R}^{n_x}$$

$$\hat{y} \in \mathbb{R}$$

$$x^{(i)} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{bmatrix}$$

$$a^{[1]} = g(w^{[1]T} X + b)$$

Diagram showing inputs (n_1, m) , (n_1, n_x) , (n_x, m) , and $(n_1, 1)$ feeding into the equation above.

$$\hat{y} = a^{[2]} = g(w^{[2]T} a^{[1]} + b)$$

Diagram showing inputs $(1, m)$, $(1, n_1)$, (n_1, m) , and $(1, 1)$ feeding into the equation above.

$$X \in \mathbb{R}^{(n_x, m)}$$

$$\hat{y} \in \mathbb{R}^m$$

$$X = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & x_1^{(3)} & \dots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & x_2^{(3)} & \dots & x_2^{(m)} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ x_{n_x}^{(1)} & x_{n_x}^{(2)} & x_{n_x}^{(3)} & \dots & x_{n_x}^{(m)} \end{bmatrix}$$

Fonction d'activation



Partie 5 : Votre premier réseau



Un modèle sans fonction d'activation

$$z^{[1]} = w^{[1]T} x + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

$$z^{[2]} = w^{[2]T} a^{[1]} + b^{[2]}$$

$$a^{[2]} = g(z^{[2]})$$

$$z^{[1]} = w^{[1]T} x + b^{[1]}$$

$$z^{[2]} = w^{[2]T} z^{[1]} + b^{[2]}$$

$$z^{[2]} = w^{[2]T} (w^{[1]T} x + b^{[1]}) + b^{[2]}$$

$$z^{[2]} = \underbrace{w^{[2]T} w^{[1]T}}_W x + \underbrace{w^{[2]T} b^{[1]} + b^{[2]}}_B$$

$$W = w^{[2]T} w^{[1]T}$$

$$B = w^{[2]T} b^{[1]} + b^{[2]}$$

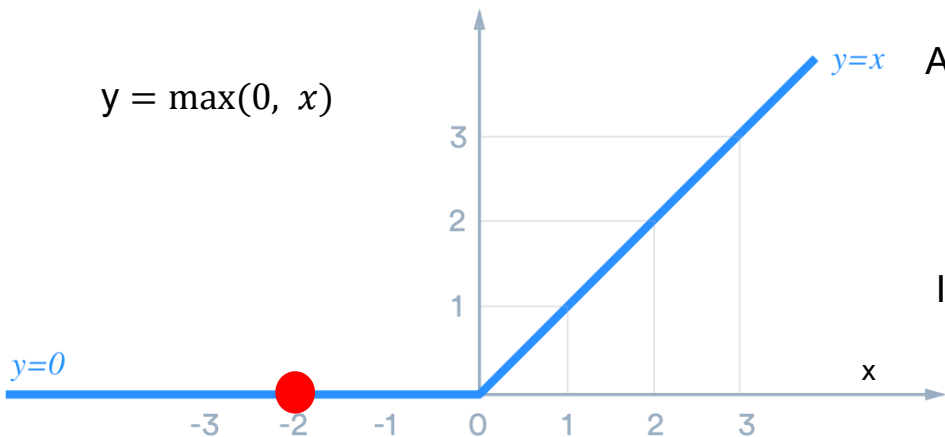
Fonction Linéaire

$$\underline{z^{[2]} = W x + B}$$



Rectified Linear Unit (ReLU)

$$y = \max(0, x)$$



Avantages :

ReLU ne sature pas pour les valeurs positives

ReLU est assez rapide à calculer

Inconvénients :

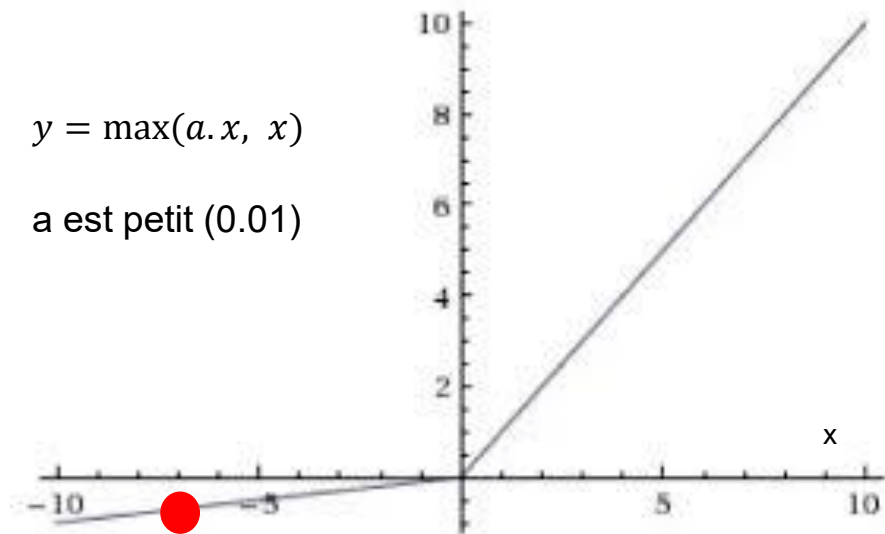
ReLU souffre d'un problème connu sous le nom de "dying ReLU".



Leaky ReLU

$$y = \max(a \cdot x, x)$$

a est petit (0.01)



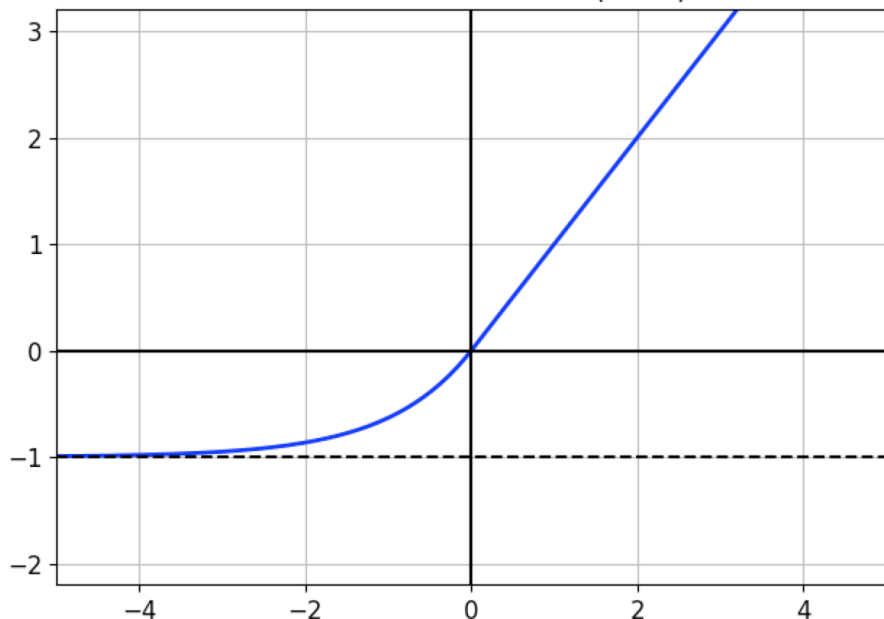
Variantes :

- Parametric leaky Relu (PReLU), si vous disposez de beaucoup de données.
- Randomized leaky Relu (RReLU), si votre réseau neuronal est surentraîné.



Exponential Linear Unit (ELU)

ELU activation function ($\alpha = 1$)



$$ELU_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

Avantages :

- Il prend des valeurs négatives, ce qui permet au neurone d'avoir une moyenne plus proche de 0
- Il a un gradient non nul pour $z < 0$, ce qui évite le problème des 'dying ReLU'.

Inconvénients :

- Il est plus lent à calculer que ReLU.



Résumé

- En général pour la performance :
$$\text{ELU} > \text{Leaky ReLU} > \text{ReLU}$$
- Si vous vous souciez du temps d'exécution :
$$\text{Leaky ReLU} > \text{ELU}$$
- Par défaut :
$$\text{ReLU}$$

La descente de gradient



Partie 5 : Votre premier réseau



Descente de gradient pour les réseaux de neurones

Paramètres : $(n^{[1]}, n^{[0]})$ $(n^{[1]}, 1)$ $(n^{[2]}, n^{[1]})$ $(n^{[2]}, 1)$ $n_x = n^{[0]}, \quad n^{[1]}, \quad n^{[2]} = 1$
 $w^{[1]}, \quad b^{[1]}, \quad w^{[2]}, \quad b^{[2]}$ $a^{[2]}$

Fonction de coût : $J(w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}) = \frac{1}{m} \sum_{i=1}^n \mathcal{L}(\hat{y}, y)$

Descente de gradient :

Répéter {

Calculer la prédiction $(\hat{y}^{(i)}, i = (1, \dots, m))$

$$\boxed{dw^{[1]}} = \frac{dJ}{dw^{[1]}}, \boxed{db^{[1]}} = \frac{dJ}{db^{[1]}}, \boxed{dw^{[2]}} = \frac{dJ}{dw^{[2]}}, \boxed{db^{[2]}} = \frac{dJ}{db^{[2]}}$$

$$w^{[1]} := w^{[1]} - \alpha dw^{[1]}$$

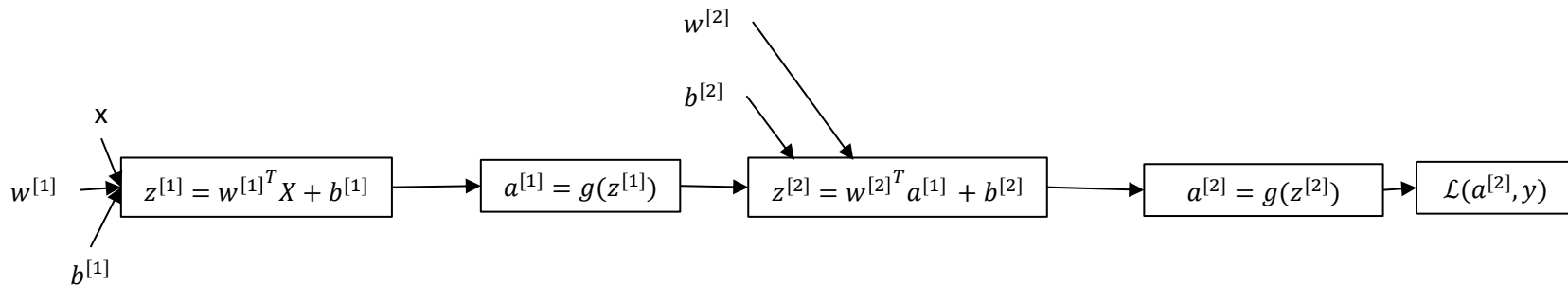
$$b^{[1]} := b^{[1]} - \alpha db^{[1]}$$

$$w^{[2]} := w^{[2]} - \alpha dw^{[2]}$$

$$b^{[2]} := b^{[2]} - \alpha db^{[2]}}$$

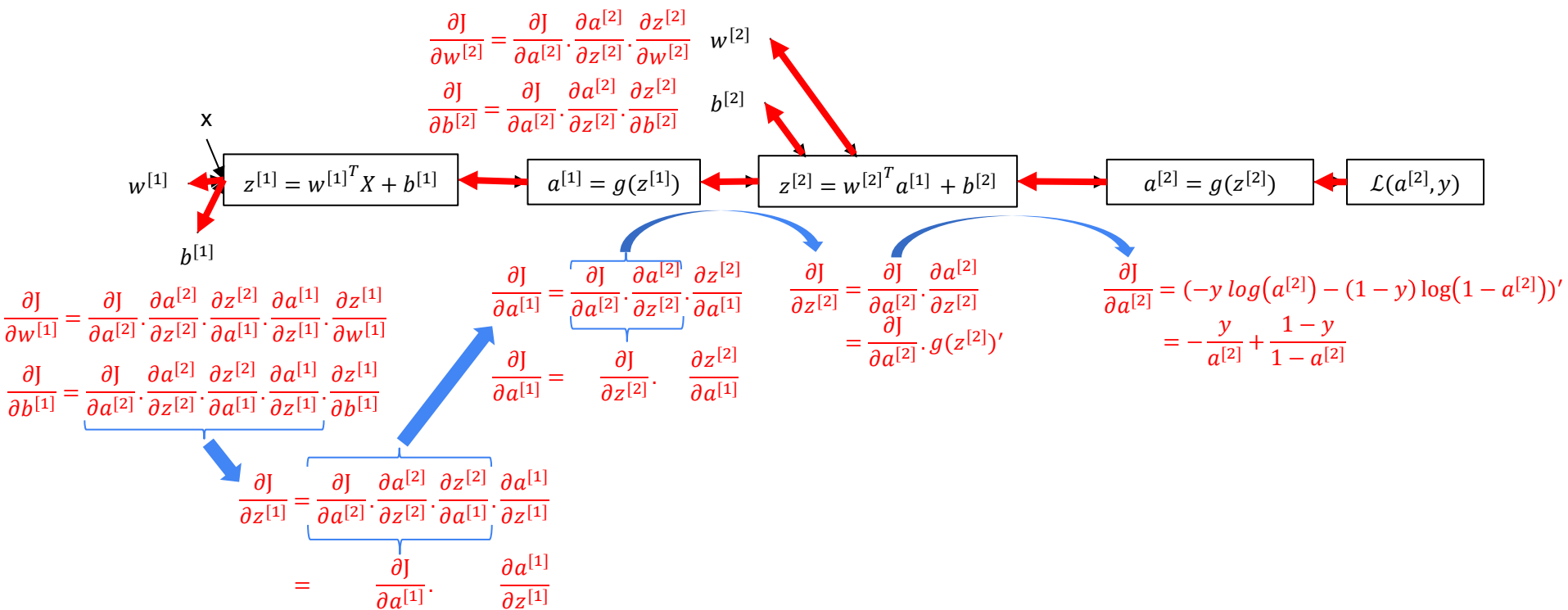


Forward propagation





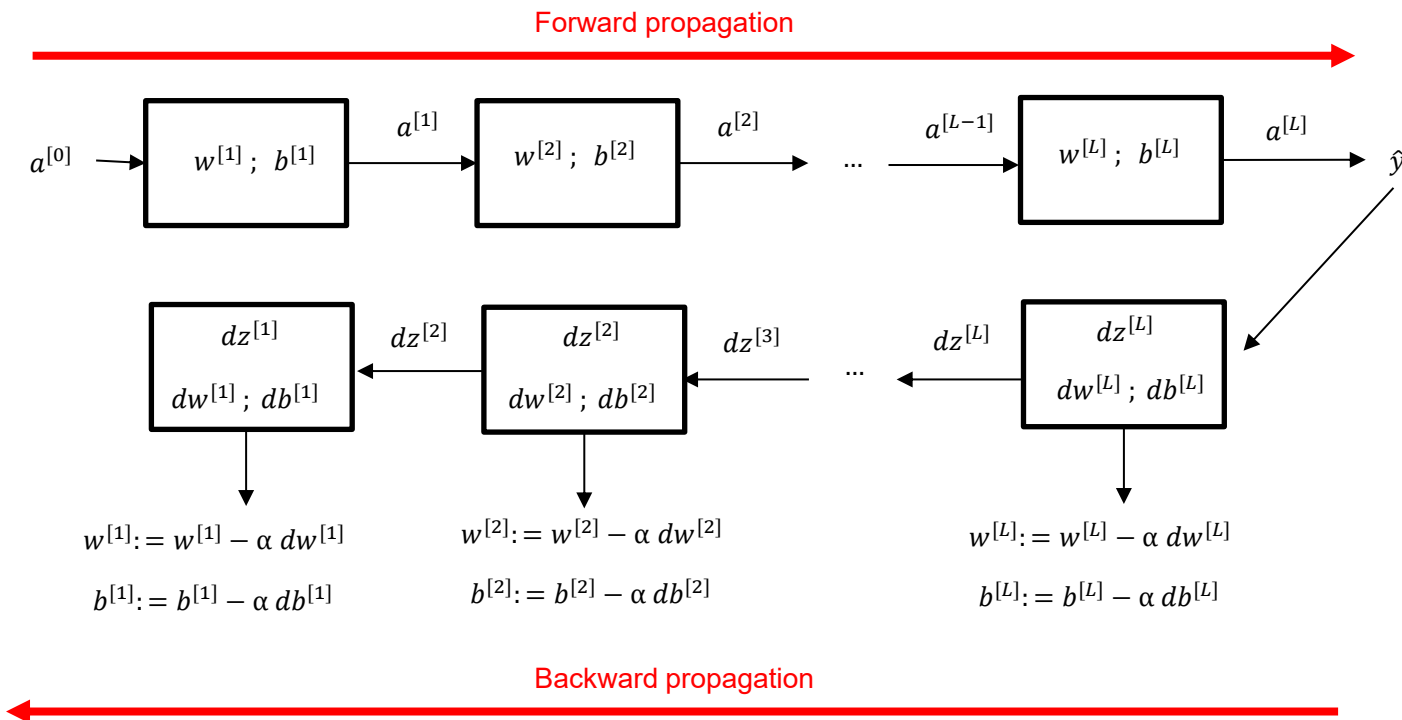
Back propagation





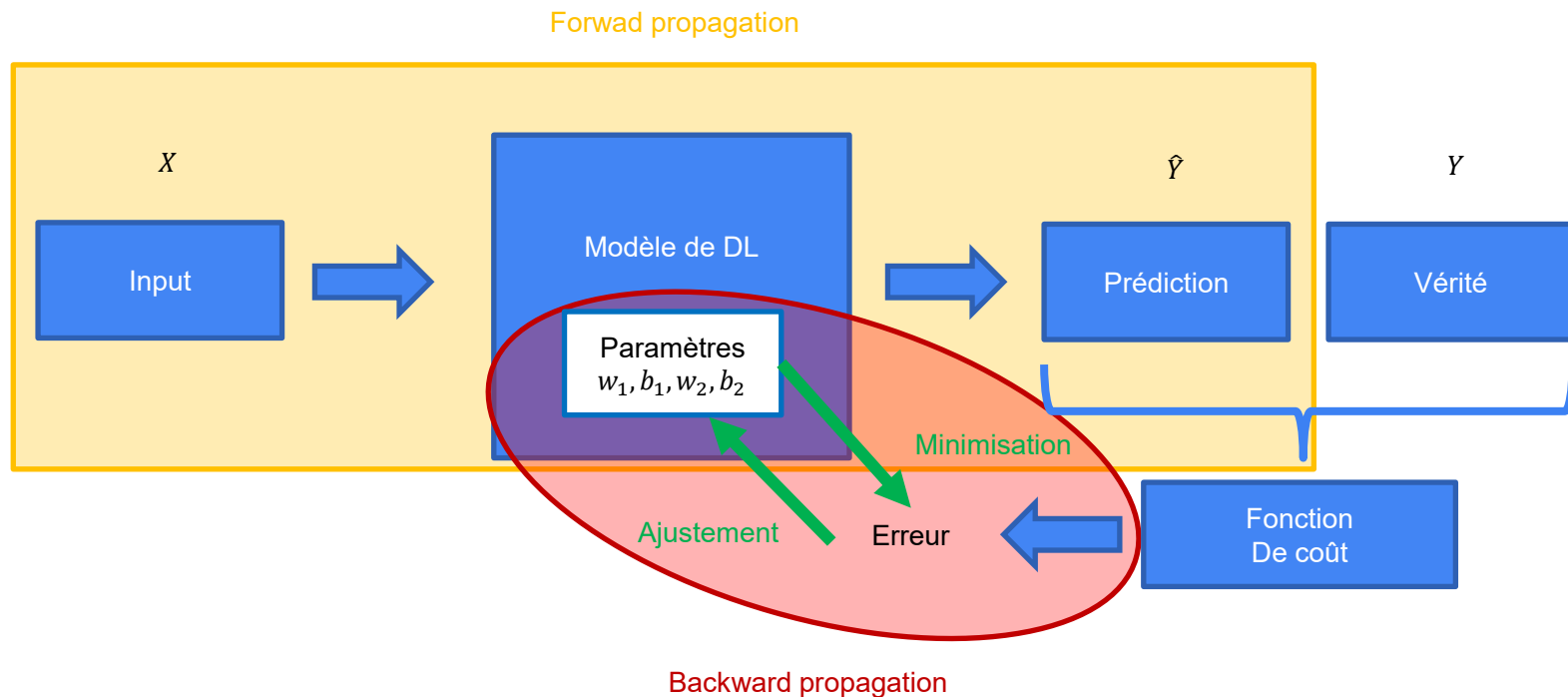
Résumé de l'apprentissage profond

$$dt = \frac{\partial J}{\partial t}$$





Entraînement d'un modèle de deep learning



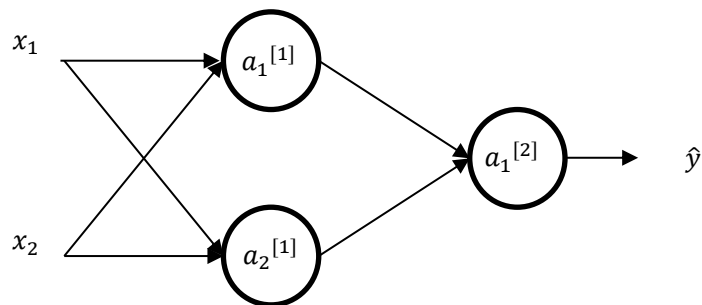
Initialisation des poids



Partie 5 : Votre premier réseau



Initialiser les poids à zéro ?



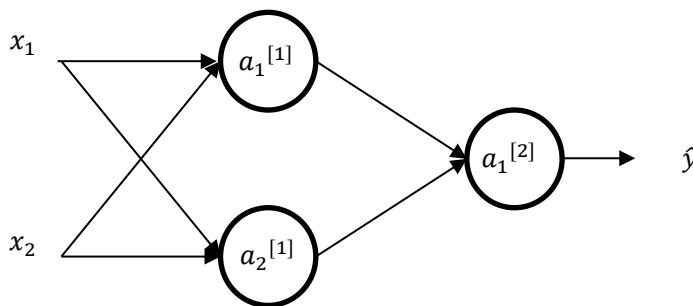
$$w^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$b^{[1]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$w^{[1]} = \begin{bmatrix} p & l \\ p & l \end{bmatrix} \leftarrow a_1^{[1]} = a_2^{[1]} \longrightarrow dz_1^{[1]} = dz_2^{[1]} \longrightarrow dw^{[1]} = \begin{bmatrix} u & v \\ u & v \end{bmatrix} \longrightarrow w^{[1]} := w^{[1]} - \alpha dw$$



Initialiser le poids de façon aléatoire



Petit nombre

$$w^{[1]} = \text{nombre_aléatoire}.0.01$$

Petit nombre

$$b^{[1]} = \text{zéro_ou_aléatoire}$$
$$w^{[2]} = \text{nombre_aléatoire}.0.01$$

$$b^{[2]} = \text{zéro_ou_aléatoire}$$

Pourquoi apprentissage profond ?



Partie 5 : Votre premier réseau



Pourquoi des représentations profondes ?

Pour approximer une fonction, une architecture de réseau neuronal plus profonde nécessite moins de paramètres qu'une architecture moins profonde.

$$y = x_1 \text{ XOR } x_2 \text{ XOR } x_3 \text{ XOR } x_4 \text{ XOR } x_5 \text{ XOR } x_6 \text{ XOR } \dots$$

